Communication Protocols

ME461 - Group: Little Daisies

Abdullah Naci Bodur Buğra Çınaroğlu Ogan Altuğ Okutan

Fall 2025-2026

Table of Contents

Serial vs. Parallel

- **Parallel Communication:**
  - Multiple wires carry multiple bits simultaneously.
  - Fast but bulky and expensive (cables, crosstalk).
- **Serial Communication:**
  - Data is sent one bit at a time over a single channel.
  - Reduces wiring complexity and cost.
  - Standard for most modern embedded systems.

Asynchronous Basics

- **No Clock Line:** Devices must agree on a timing speed (*Baud Rate*) beforehand.
- **Synchronization:** Done via "Start" and "Stop" bits.
- **Common Protocols:** UART (Logic Level), RS-232 (Voltage Level).

UART: Working Principle

Universal Asynchronous Receiver-Transmitter (UART)

1. **Parallel-to-Serial:** CPU writes byte to Transmit Register → Shift Register pushes bits out one by one.
2. **Serial-to-Parallel:** Receiver samples line → shifts bits in → CPU reads byte.

- **Wiring:** Cross-coupled (TX connects to RX).
- **Topology:** Strictly Point-to-Point (1 Master, 1 Slave).

UART Frame Structure

Standard 8N1 Frame (8 data bits, No parity, 1 stop bit):

```
Idle (High)
   |
   v
 Start | D0  D1  D2  D3  D4  D5  D6  D7 | Stop
  (0)  |           DATA                 | (1)
```

- **Start Bit:** Pulls line LOW (0) to wake up receiver.
- **Data:** Sent LSB (Least Significant Bit) first.
- **Parity (Optional):** Error checking bit.
- **Stop Bit:** Pulls line HIGH (1) to reset for next byte.

ME461 Application: UART Scope Project

Project Context:

- Used for **Raspberry Pi Pico programming** (REPL) and the **Scope Project**.
- The Pico reads signals/PWM and transmits data to the PC for visualization.

Configuration:

- **Baud Rate:** 115200
- **Connection:** UART-over-USB or physical UART pins.

Pico UART Pinout

Function	Pin
TX	GP0
RX	GP1
GND	Any GND

**Scope project sends PWM values to Python script via this link.*

RS-232: The Physical Layer

UART is a logic protocol; RS-232 is the voltage standard often used with it.

- **Purpose:** Long distance communication (up to 15m) where 5V signals would degrade.
- **Voltage Levels (Inverted Logic):**
  - Logic 0: +3V to +15V
  - Logic 1: -3V to -15V

```
      +15V  ------+        +--------
Logic 0           |        |
      -15V        +--------+
Logic 1
```

Note: You need a MAX232 converter to connect this to a microcontroller.

SPI: Serial Peripheral Interface

Overview:

- Synchronous: Uses a dedicated clock line (SCK).
- Architecture: Master-Slave.
- Duplex: **Full Duplex** (Send and Receive simultaneously).

Wiring (4 Lines):

1. **SCK:** Serial Clock (generated by Master).
2. **MOSI:** Master Out Slave In (Data to slave).
3. **MISO:** Master In Slave Out (Data to master).
4. **CS/SS:** Chip Select (Active Low, selects the specific slave).

SPI Topology

- **Standard Mode:** One CS line per Slave.
- **Daisy Chain:** Slaves connected in series (Output of one goes to Input of next).

Advantages:

- Fastest serial protocol (10MHz - 100MHz).
- No complex addressing (hardware selection via CS).

Disadvantages:

- Requires many wires (3 + N slaves).
- No hardware acknowledgment (blind transmission).

SPI Timing Modes

SPI has 4 modes based on Clock Polarity (CPOL) and Phase (CPHA).

- **CPOL:** Idle state of clock (High or Low).
- **CPHA:** Data sampled on Rising or Falling edge.

```
Mode 0 (Standard):
SCK:    _   _   _   _
       / \_/ \_/ \_/ \_  (Sample on Rising Edge)
MOSI:  Valid Data
```

ME461 Application: SPI Tetris Game

Project Context:

- Used in the **Tetris Game** project.
- High-speed communication required to update the LED grid matrix smoothly.

Hardware:

- **Master:** Raspberry Pi Pico
- **Slave:** MAX7219 Dot Matrix Driver

Pico SPI0 Pinout

Function	**Pin**
MOSI	GP19
SCK	GP18
CS (SS)	GP17
VCC	5V (VBUS)

**MISO is not used here (Display only receives data).*

I^2C: Inter-Integrated Circuit

Overview:

- Synchronous, Half-Duplex.
- **2 Wires Only:** SDA (Data) and SCL (Clock).
- **Addressed Based:** No Chip Select lines; devices have a 7-bit address.

Key Features:

- Multiple Masters and Multiple Slaves allowed.
- **Open Drain:** Requires Pull-up Resistors (4.7kΩ).
- **Clock Stretching:** Slave can hold SCL low to pause Master.

I^{2}C Frame Structure

[Image of I2C data packet structure]

```
[Start][Address (7bit)] [R/W] [ACK] [Data (8bit)] [ACK][Stop]
```

1. **Start Condition:** SDA goes LOW while SCL is HIGH.
2. **Address:** Master broadcasts target address.
3. **ACK/NACK:** Receiver pulls SDA low to acknowledge receipt.
4. **Stop Condition:** SDA goes HIGH while SCL is HIGH.

I^{2}C Pros and Cons

Advantages:

- Low pin count (just 2 pins for up to 127 devices).
- Hardware flow control (ACK/NACK) ensures data integrity.
- Standard for sensors (IMUs, Temp sensors, EEPROMs).

Disadvantages:

- Slower than SPI (Standard: 100kHz, Fast: 400kHz).
- Complex software implementation compared to SPI.
- Pull-up resistors consume power.

ME461 Application: I2C Mechaboard

Project Context:

- Used on the **Mechaboard** platform.
- **Bus Sharing:** Two different sensors are controlled using the same single pair of I2C pins.

Devices on Bus:

1. **MPU6050:** IMU (Accel/Gyro)
2. **SSD1306:** OLED Display

Pico I2C0 Pinout

Function	Pin
SDA	GP4
SCL	GP5
VCC	3.3V

**Different addresses allow both chips to talk on GP4/GP5 simultaneously.*

Comparison Matrix

Feature	**UART / RS-232**	**SPI**	**I^2C**
Wires	2 (TX, RX)	4 (SCK, MOSI, MISO, CS)	2 (SDA, SCL)
Synchronous	No (Async)	Yes	Yes
Speed	Slow (¡115 kbps typical)	Fastest (¿10 Mbps)	Medium (100-400 kbps)
Topology	Point-to-Point	Single Master, Multi-Slave	Multi-Master, Multi-Slave
Flow Control	None / Software	None	ACK/NACK
Duplex	Full Duplex	Full Duplex	Half Duplex
Best For	PC $\leftrightarrow$ MCU, Debugging	Screens, SD Cards	Sensors, Low Pin Count

Conclusion

How to choose?

- Need simple PC logging or long distance? $\rightarrow$ **UART / RS-232**
- Need high speed data transfer (Display, Flash Memory)? $\rightarrow$ **SPI**
- Need many sensors with few pins? $\rightarrow$ **I^2C**

Questions?